

Decision making in java

- Decision making in java helps to write decision driven statements and execute a particular set of code based on certain conditions.

1. If Statement

- The java if statement is the most simple decision making statement.
- It is used to decide whether a certain statement or block of statements will be execute or not.
- The Java if statement tests the condition. It executes the *if block* if condition is true.

Syntax:

- **if**(condition){
 //code to be executed
}

2. if...else statement

- This pair of keywords is used to divide a program to be executed into two parts, one being the code to be executed if the condition evaluates to true and the other one to be executed if the value is false.
- However if no curly braces are given the first statement after the if or else keyword is executed.

- Syntax:

- if(condition)
 {
 //code to be executed if the condition is true
 }
 else
 {
 //code to be executed if the condition is false
 }

3. Nested If

- The nested if is similar to the nested loops we learnt about in the previous chapters. If the condition of the outer if statement evaluates to true then the inner if statement is evaluated.
- Nested if's are important if we have to declare extended conditions to a previous condition.

Syntax:

- `if(condition)`
 `{`
 `//code to be executed`
 `if(condition)`
 `{`
 `//code to be executed`
 `}`
 `}`

4. if-else-if Statement

- These statements are similar to the if else statements .
- The only difference lies in the fact that each of the else statements can be paired with a different if condition statement. This renders the ladder as a multiple choice option for the user.
- soon as one of the if conditions evaluates to true the equivalent code is executed and the rest of the ladder is ignored.

Syntax:

- ```
if
{
//code to be executed
}
else if(condition)
{
//code to be executed
}
else if(condition)
{
//code to be executed
}
else
{
//code to be executed
}
```

# 5. Switch Statement

- Java switch statement is a different form of the if else if ladder statements.
- This saves the hassle of writing else if for every different option.
- It branches the flow of the program to multiple points as and when required or specified by the conditions.
- It bases the flow of the program based on the output of the expression.

- As soon as the expression is evaluated, the result is matched with each and every case listed in the scope. If the output matches with any of the cases mentioned, then the particular block is executed. A break statement is written after every end of the case block so that the remaining statements are not executed.
- The default case is written which is executed if none of the cases are the result of the expression. This is generally the block where error statements are written.

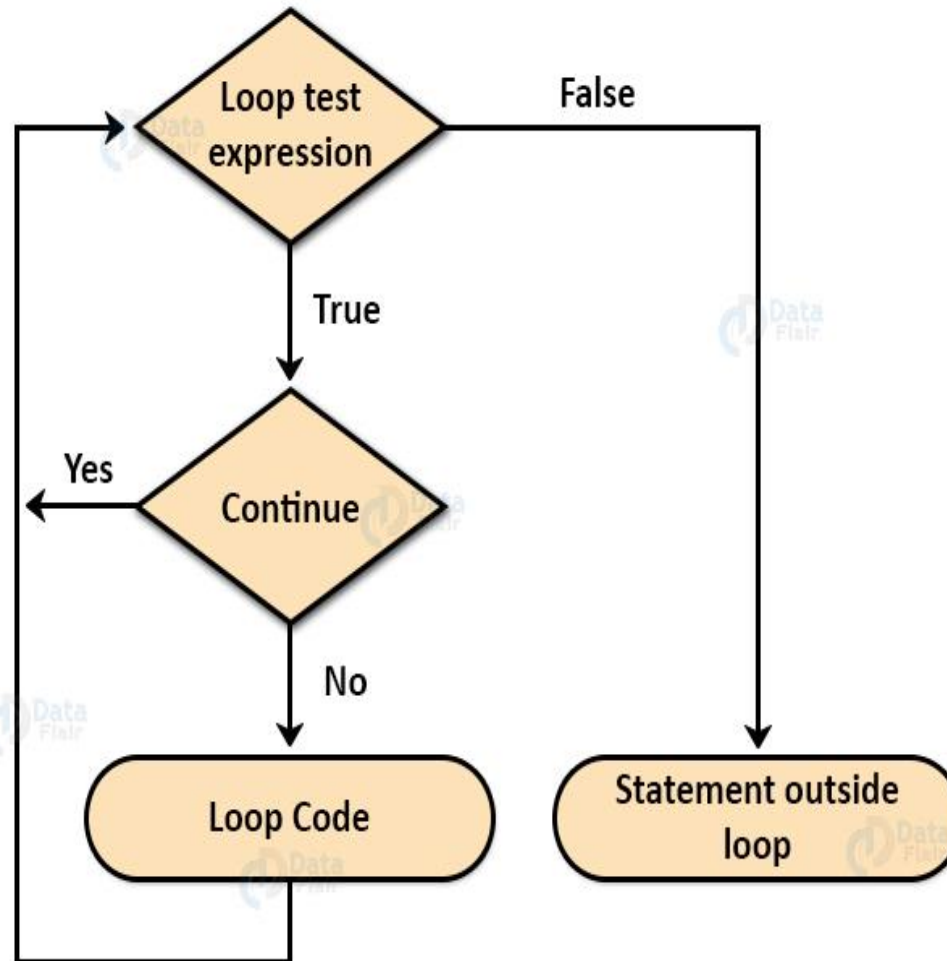
# Syntax:

- `switch(expression)`  
  {  
    case <value1>:  
      //code to be executed  
      break;  
    case <value2>:  
      //code to be executed  
      break;  
    default:  
      //code to be defaultly executed  
  }

## 6. Continue statement

- The continue statement is useful for early iteration of a particular loop.
- Sometimes rather than breaking out of a loop and halting it for good we want to skip some iterations based on our requirements of a program.
- This results in the usage of continue statements.

## Flow Diagram for the continue statement



# 7. Break statement

- If we mention a label after the break keyword, the control shifts to the end of the particular label scope.
- Java prohibits the use of goto statements because it encourages branched behaviour of programming. However break labels function like goto statements.

## Flowchart of break statement

